

DBT Day 2

Shivam Zanjurne · Fri, 09 Oct 26 · ALTER, INSERT...SELECT, DELETE/TRUNCATE/DROP, TCL, PK + AUTO_INCREMENT, DEFAULT, NULL, temp tables, WHERE operators, MySQL data types

Table of Contents

- | | |
|---|--|
| 1. Workflow: CMD, Workbench and Notepad | 10. DEFAULT Column Values |
| 2. Reading DESCRIBE Output | 11. What NULL Really Means |
| 3. INSERT ... SELECT — Copy Rows Between Tables | 12. Temporary Tables |
| 4. ORDER BY and Column Display Order | 13. UPDATE and DELETE with WHERE |
| 5. DDL Recap and ALTER TABLE | 14. WHERE Clause: Comparison and Logical Operators |
| 6. DROP TABLE and DROP DATABASE | 15. How MySQL Stores Data Internally |
| 7. DELETE vs TRUNCATE vs DROP | 16. MySQL Data Types |
| 8. TCL — Transactions, Savepoints, Commit | 17. Day Wrap and Tomorrow |
| 9. PRIMARY KEY and AUTO_INCREMENT | |
-

Workflow: CMD, Workbench and Notepad

Exam and this module prefer **CMD**. Workbench is fine for practice, but auto-suggest can hide whether you really know the SQL.

CMD vs Workbench vs industry tools

Tool	Role
CMD (<code>mysql -u root -p</code>)	Primary for class + exams — you type every keyword yourself
MySQL Workbench	GUI; same SQL, but suggestions make learning weaker
pgAdmin	GUI for PostgreSQL (mentioned for awareness)

Industry often uses a CLI or thin client against the server; GUIs exist when needed.

Writing queries in Notepad

You **can** write multi-line SQL in Notepad, copy, and paste into CMD. CMD executes the full pasted block (not only the last line).

Caveat: if the paste breaks mid-statement (arrow keys / incomplete paste), you may need to re-paste the whole command.

```
CREATE TABLE demo (  
  id      INT,  
  name    VARCHAR(20),  
  email_id VARCHAR(20)  
);  
  
INSERT INTO demo (id, name, email_id) VALUES  
  (1, 'Aniket', 'aniket@gmail.com'),  
  (2, 'Sahil', 'sahil@gmail.com'),  
  (3, 'Yash', 'yash@gmail.com');  
  
SELECT * FROM demo;
```

Workbench tips from class

- **Ctrl + Enter** — run the statement under the cursor.
- **Safe Updates** blocks `UPDATE` / `DELETE` without a key in `WHERE`. To allow unrestricted deletes for practice:
 1. Edit → Preferences → **SQL Editor**
 2. Uncheck **Safe Updates (rejects UPDATES and DELETES with no restrictions)**
 3. Reconnect / restart the SQL tab if needed

Prefer practicing dangerous DML in CMD with transactions (Section 8) so you can `ROLLBACK`.

Reading DESCRIBE Output

```
DESCRIBE student;  
-- or: DESC student;
```

Column in DESCRIBE	Meaning
Field	Column name
Type	Data type + size
Null	<code>YES</code> = NULL allowed; <code>NO</code> = NOT NULL
Key	<code>PRI</code> = primary key, <code>UNI</code> = unique, <code>MUL</code> = indexed / part of a key
Default	Default value if insert omits the column
Extra	Extras such as <code>auto_increment</code>

The first column is **not** automatically a primary key. You must declare `PRIMARY KEY` (or add it later with `ALTER`).

INSERT ... SELECT — Copy Rows Between Tables

Why not copy row-by-row?

A master table (e.g. registration with 10+ columns and thousands of rows) is awkward for every small task. Class pattern:

1. Create a **narrow** table with only the columns you need.
2. Fill it with `INSERT ... SELECT` from the master.
3. Run your work on the narrow table.

Create a backup / subset table

```
CREATE TABLE student_backup (
  id      INT,
  name    VARCHAR(20),
  course  VARCHAR(20)
);
```

Copy all selected columns

```
INSERT INTO student_backup
SELECT id, name, course
FROM student;
```

`SELECT` alone only **displays**. Wrapping it in `INSERT INTO ...` **writes** those rows into the target.

Copy only some columns (target has more columns)

If `student_backup` has three columns but you only want to fill two, name the target columns:

```
INSERT INTO student_backup (id, name)
SELECT id, name
FROM student;
```

Filter while copying — WHERE

```
TRUNCATE student_backup;           -- empty it first (or DELETE FROM student_backup)

INSERT INTO student_backup
SELECT id, name, course
FROM student
WHERE id > 5;
```

You can run further inserts with different filters (e.g. `WHERE id < 5`). Rows accumulate unless you truncate/delete first. Existing data in the target is fine — new rows are **appended**.

Exam idea: “Import a fat CSV into a master table, then project only the columns your feature needs into a working table.”

ORDER BY and Column Display Order

Sort result rows

Default insert order is not a guaranteed sorted display. Sort explicitly:

```
SELECT * FROM student_backup
ORDER BY id;           -- ascending (ASC is default)

SELECT * FROM student_backup
ORDER BY name ASC;

SELECT * FROM student_backup
ORDER BY course DESC;
```

Display column order ≠ physical table order

Physical column order is fixed by `CREATE` / `ALTER`. For **output**, list columns in the order you want:

```
SELECT course, name, id
FROM student_backup;
```

First column shown is `course` , then `name` , then `id` — even if the table was created as `id, name, course` .

DDL Recap and ALTER TABLE

DDL commands (structure)

Command	Effect
<code>CREATE</code>	Create database / table
<code>ALTER</code>	Change existing structure
<code>DROP</code>	Remove table / database (structure + data)
<code>TRUNCATE</code>	Remove all rows ; keep structure

`ALTER` is **DDL**. With it you can: **ADD, MODIFY, CHANGE, DROP** columns; also **RENAME** columns / tables.

ADD a column

```
ALTER TABLE student
ADD COLUMN grade CHAR(5);

ALTER TABLE student
ADD COLUMN email_id VARCHAR(5);  -- class demo: size too small on purpose
```

```
DESC student;  -- grade / email_id appear at the end
```

MODIFY — change data type / size (same column name)

```
ALTER TABLE student
MODIFY email_id VARCHAR(30);
```

Use when the name stays the same and only the type/size must grow (e.g. emails need more than 5–20 chars).

CHANGE — rename column and/or change type

```
ALTER TABLE student
CHANGE name student_name VARCHAR(30);
```

Old name → new name + new definition in one statement.

RENAME COLUMN (MySQL 8+)

```
ALTER TABLE student
RENAME COLUMN student_age TO age;
```

DROP a column

```
ALTER TABLE student
DROP COLUMN grade;
```

The column **and all its values** are gone. Dropping `course` removes every course value for every row.

RENAME a table

```
RENAME TABLE student TO pgcp_student;

SHOW TABLES;  -- student gone; pgcp_student present
```

Case sensitivity notes (Windows MySQL)

Object	Behaviour in class
Table names	Usually folded to lowercase in <code>SHOW TABLES</code> (<code>Student2</code> → <code>student2</code>)
Column names	Stored as you defined them; referring with a different mid-name casing can cause <code>Unknown column</code>
SQL keywords	Case-insensitive (<code>SELECT</code> = <code>select</code>)

Prefer consistent lowercase snake_case: `student_name` , `email_id` . Avoid spaces in names — use `_` instead.

Naming tip from class

Do **not** put spaces in column names. Use `student_name` , not ``student name`` .

DROP TABLE and DROP DATABASE

Drop a table

```
DROP TABLE demo;  
  
SHOW TABLES;    -- demo gone
```

Avoid errors with IF EXISTS

```
DROP TABLE demo;           -- ERROR if demo already gone: Unknown table ...  
  
DROP TABLE IF EXISTS demo; -- no error; 0 rows affected / warning only
```

Same idea for databases:

```
DROP DATABASE IF EXISTS august26;
```

DELETE vs TRUNCATE vs DROP

DELETE (DML)

```
DELETE FROM student_backup  
WHERE id = 3;           -- one row  
  
DELETE FROM student_backup; -- ALL rows if no WHERE - dangerous
```

- Can use **WHERE**
- Table **structure remains**
- Can **ROLLBACK** if inside a transaction (InnoDB)
- Deletes **row by row** → slower on huge tables
- **AUTO_INCREMENT** is **NOT** reset
- Can fire **triggers** (ON DELETE) — covered later in depth

TRUNCATE (DDL)

```
TRUNCATE TABLE student_backup;
```

- Deletes **all** rows in one shot — **no** **WHERE**

- Structure **remains**
- **Cannot rollback** like a normal DML delete (DDL; auto-committed)
- **Faster** than deleting all rows with `DELETE`
- `AUTO_INCREMENT` **is reset** to start again from 1
- Triggers on `DELETE` do **not** apply the same way as row-by-row delete

DROP (DDL)

```
DROP TABLE student_backup;
```

- Removes **data + structure**
- Table is gone — you must `CREATE` again to reuse the name
- No `WHERE` ; no rollback of the drop itself
- No meaningful “table left behind” for further DML

Comparison table (memorize)

Point	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	Selected / all rows	All rows	Table (rows + structure)
<code>WHERE</code>	Yes	No	No
Structure kept?	Yes	Yes	No
Rollback (typical InnoDB txn)	Yes	No (auto-commit)	No
<code>AUTO_INCREMENT</code> reset?	No	Yes	N/A (table gone)
Speed (clear all rows)	Slower	Faster	Fast (gone)
Triggers	Can fire per row	Not like <code>DELETE</code>	N/A

AUTO_INCREMENT reset demo (class idea)

```
-- After many inserts, last id was 11
DELETE FROM employee;           -- all rows gone, but next insert → 12

TRUNCATE TABLE employee;       -- all rows gone, next insert → 1 again
```

TCL — Transactions, Savepoints, Commit

TCL = Transaction Control Language: `START TRANSACTION` , `SAVEPOINT` , `ROLLBACK` , `COMMIT` .

Works with DML only

Can use TCL	Cannot undo with TCL
<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code>

DDL is effectively **auto-committed** by the storage engine. Once `ALTER` / `DROP` runs, you cannot `ROLLBACK` to the old structure.

```
SELECT @@autocommit;    -- often 1 (ON) by default
```

Why start a transaction before UPDATE / DELETE?

Without `WHERE` , `DELETE` or `UPDATE` hits **every row**. Inside a transaction you can undo:

```
START TRANSACTION;

DELETE FROM student WHERE id = 2;
SELECT * FROM student;           -- id 2 gone

ROLLBACK;                        -- bring id 2 back
SELECT * FROM student;
```

Habit: **before** risky `UPDATE` / `DELETE` , `START TRANSACTION` . Verify with `SELECT` . Then `COMMIT` or `ROLLBACK` .

SAVEPOINT — partial undo

A savepoint is a named checkpoint inside an open transaction.

```
START TRANSACTION;

SAVEPOINT s1;
INSERT INTO student (id, student_name, age, course, city, email_id)
VALUES (90, 'Samit', 30, 'MSC', 'Pune', 'samit@mail.com');

SAVEPOINT s2;
INSERT INTO student (id, student_name, age, course, city, email_id)
VALUES (91, 'Yash', 28, 'BSC', 'Nagpur', 'yash@mail.com');

ROLLBACK TO s2;    -- removes Yash insert; Samit still there
ROLLBACK TO s1;    -- removes Samit too (back to state at s1)
```

- ## COMMIT — make changes permanent

After COMMIT :

- ```

START TRANSACTION
|
| SAVEPOINT s1 — work A — SAVEPOINT s2 — work B
| |
| | ROLLBACK TO s1 ROLLBACK TO s2
| | (undo A and B) | (undo B only)
|
| COMMIT → all remaining changes permanent; txn closed

```

Large maintenance scripts: take a savepoint after each successful chunk. If chunk 3 fails, `ROLLBACK TO` the savepoint after chunk 2 instead of restoring the whole database from backup.

# PRIMARY KEY and AUTO\_INCREMENT

## Primary key properties

| Rule             | Meaning                                                         |
|------------------|-----------------------------------------------------------------|
| Unique           | No two rows share the same PK value                             |
| NOT NULL         | You cannot insert <code>NULL</code> into the PK                 |
| One PK per table | May be one column or a composite of several                     |
| Index            | MySQL creates an index on the PK automatically → faster lookups |

The first column named `id` is **not** a PK unless you declare it.

## Create with PK + AUTO\_INCREMENT

```
CREATE TABLE employee (
 emp_id INT PRIMARY KEY AUTO_INCREMENT,
 emp_name VARCHAR(25),
 dept_no INT,
 salary DECIMAL(10, 2),
 email_id VARCHAR(30)
);

DESC employee;
-- Key = PRI, Extra = auto_increment for emp_id
```

## Insert without giving the id

```
INSERT INTO employee (emp_name, dept_no, salary, email_id) VALUES
 ('Aniket', 20, 50000, 'aniket@mail.com'),
 ('Sahil', 25, 55000, 'sahil@mail.com'),
 ('Gauri', 30, 60000, 'gauri@mail.com');

SELECT * FROM employee; -- emp_id 1, 2, 3 assigned automatically
```

## Explicit id jumps the counter

```
INSERT INTO employee (emp_id, emp_name, dept_no, salary, email_id)
VALUES (10, 'Vishal', 25, 60000, 'vishal@mail.com');

-- Next auto value continues AFTER 10 (e.g. 11), not after 3
INSERT INTO employee (emp_name, dept_no, salary, email_id)
VALUES ('Shilpa', 20, 30000, 'shilpa@mail.com');
```

## Duplicate PK error

```
INSERT INTO employee (emp_id, emp_name, dept_no, salary, email_id)
VALUES (10, 'Someone', 20, 40000, 'x@mail.com');
```

```
ERROR 1062 (23000): Duplicate entry '10' for key 'employee.PRIMARY'
```

## Add a PK later with ALTER (lab practice)

```
ALTER TABLE some_table
ADD PRIMARY KEY (id);
```

Do **not** put `DEFAULT` on a primary key — uniqueness would break if every omitted insert got the same default.

## DEFAULT Column Values

When a column is omitted from `INSERT` and has a `DEFAULT`, MySQL fills that value.

```
CREATE TABLE department (
 dept_id INT PRIMARY KEY,
 dept_name VARCHAR(25),
 salary INT DEFAULT 25000,
 location VARCHAR(20) DEFAULT 'Kharghar'
);

DESC department; -- Default column shows 25000 / Kharghar
```

## Insert with all values

```
INSERT INTO department (dept_id, dept_name, salary, location) VALUES
(1, 'Education', 50000, 'Juhu'),
(2, 'Research', 60000, 'Bandra');
```

## Insert omitting columns that have defaults

```
INSERT INTO department (dept_id, dept_name) VALUES
 (3, 'Training'),
 (4, 'Testing');

SELECT * FROM department;
-- Training / Testing get salary=25000, location='Kharghar'
```

## What NULL Really Means

**NULL** means the value is **unknown / not provided**.

| Wrong idea               | Reality                                      |
|--------------------------|----------------------------------------------|
| NULL = 0                 | No — 0 is a known number                     |
| NULL = '' (empty string) | No — empty string is a known (empty) value   |
| NULL = "missing forever" | It means "we don't have a value <b>now</b> " |

```
INSERT INTO employee (emp_name, dept_no, salary)
VALUES ('Shivraj', 20, 45000);
-- email_id not listed, no DEFAULT → email_id is NULL
```

Compare NULLs with **IS NULL** / **IS NOT NULL**, never **= NULL** (Section 14).

## Temporary Tables

### What and why

A **temporary table** lives for the **current session** only. When you disconnect (or drop it), it disappears. Data you already **INSERT** ed into a **normal** table from the temp table stays.

**Uses (class):**

1. **Break complex queries** — extract intermediate results from many tables (e.g. ~70 tables in a real project).
2. **Performance** — work on a small extracted set instead of joining everything repeatedly.
3. **Safe sandbox** — experiment with **UPDATE** / **DELETE** without touching the master table.
4. **Session-specific state** — scratch space for this connection only.

## Syntax

```
CREATE TEMPORARY TABLE employee_temp (
 emp_id INT,
 emp_name VARCHAR(25),
 salary INT,
 dept_name VARCHAR(25)
);

INSERT INTO employee_temp (emp_id, emp_name, salary, dept_name) VALUES
 (1, 'Aniket', 60000, 'Education'),
 (2, 'Ritesh', 75000, 'Training');

SELECT * FROM employee_temp;

DROP TEMPORARY TABLE employee_temp; -- optional; auto-dropped on logout
```

Keyword is **TEMPORARY**. After session end, the temp table is gone even if you never **DROP** ped it.

## UPDATE and DELETE with WHERE

### UPDATE pattern

```
UPDATE student
SET age = age + 2
WHERE id = 6;
```

Raise salary for one department (idea from class):

```
UPDATE employee
SET salary = salary * 1.20
WHERE dept_name = 'Education'; -- only that department
```

### DELETE pattern

```
DELETE FROM employee
WHERE emp_id = 10;
```

## Critical rule

| Statement                                 | Without WHERE                         |
|-------------------------------------------|---------------------------------------|
| <code>UPDATE student SET age = 30;</code> | <b>Every</b> student's age becomes 30 |
| <code>DELETE FROM student;</code>         | <b>Every</b> row deleted              |

Always:

1. `START TRANSACTION;`
2. Run `UPDATE` / `DELETE` with `WHERE`
3. `SELECT` to verify
4. `COMMIT;` or `ROLLBACK;`

---

## WHERE Clause: Comparison and Logical Operators

`WHERE` **restricts rows** for `SELECT`, `UPDATE`, and `DELETE` (DML / queries — not for changing table structure).

### Comparison operators

| Operator                                | Meaning          | Example                             |
|-----------------------------------------|------------------|-------------------------------------|
| <code>=</code>                          | Equal            | <code>course = 'Java'</code>        |
| <code>≠</code> or <code>&lt;&gt;</code> | Not equal        | <code>course &lt;&gt; 'Java'</code> |
| <code>&gt;</code>                       | Greater than     | <code>age &gt; 30</code>            |
| <code>≥</code>                          | Greater or equal | <code>age ≥ 30</code>               |
| <code>&lt;</code>                       | Less than        | <code>id &lt; 5</code>              |
| <code>≤</code>                          | Less or equal    | <code>age ≤ 30</code>               |

```

SELECT * FROM pgcp_student
WHERE course = 'Java';

SELECT * FROM pgcp_student
WHERE course <> 'Java'; -- same idea as ≠

SELECT * FROM pgcp_student
WHERE age > 30;

SELECT * FROM pgcp_student
WHERE age ≥ 30;

SELECT * FROM pgcp_student
WHERE age < 30;

SELECT * FROM pgcp_student
WHERE age ≤ 30;

```

## Logical operators — AND, OR, NOT

| Operator | Row kept when...                   |
|----------|------------------------------------|
| AND      | <b>All</b> conditions true         |
| OR       | <b>At least one</b> condition true |
| NOT      | Condition is <b>not</b> true       |

```

-- Both must match
SELECT * FROM pgcp_student
WHERE age ≤ 30 AND course = 'MySQL';

-- Either may match
SELECT * FROM pgcp_student
WHERE age = 27 OR course = 'Java';

-- Multiple ANDs fine
SELECT * FROM pgcp_student
WHERE age = 27
 AND email_id IS NOT NULL
 AND city = 'Pune';

```

## Parentheses with AND + OR

AND binds tighter than OR in practice — **use parentheses** so intent is obvious:

```
SELECT * FROM pgcp_student
WHERE age < 32 OR city = 'Pune' AND course = 'Java';
-- easy to misread

SELECT * FROM pgcp_student
WHERE (age < 32 OR city = 'Pune') AND course = 'Java';
-- clear: must be Java, and (young or in Pune)
```

## IS NULL / IS NOT NULL

```
-- WRONG - never true in SQL three-valued logic
SELECT * FROM employee WHERE email_id = NULL;

-- RIGHT
SELECT * FROM employee WHERE email_id IS NULL;
SELECT * FROM employee WHERE email_id IS NOT NULL;
```

## IN / NOT IN — membership lists

Instead of many `OR` s:

```
SELECT * FROM pgcp_student
WHERE course IN ('Java', 'MySQL', '.NET');

-- equivalent idea:
-- WHERE course = 'Java' OR course = 'MySQL' OR course = '.NET'

SELECT * FROM pgcp_student
WHERE id IN (1, 2, 3, 4, 5, 6, 7);

SELECT * FROM pgcp_student
WHERE course NOT IN ('Java', '.NET');
```

Works the same spirit with `UPDATE` / `DELETE` `WHERE` clauses.

---

## How MySQL Stores Data Internally

UI shows neat rows and columns. Internally, InnoDB organises data differently.

## Layers (simplified)

```
Database
├── Tablespace
│ ├── Pages (default 16 KB each) ← managed by InnoDB
│ └── Records / rows / tuples
```

- One table with 10 lakh rows spans **many pages**.
- Current MySQL default engine: **InnoDB**.

## Buffer pool (RAM) vs disk

```
Query fired
 |
 ▼
Is needed page in BUFFER POOL (RAM)?
├── YES → use it (fast)
└── NO → read page from DISK → put into buffer pool → use it
```

| Location          | Role                                            |
|-------------------|-------------------------------------------------|
| Disk              | Permanent storage of the database               |
| Buffer pool (RAM) | Cache of <b>frequently used pages</b> for speed |

Disk I/O is slow; keeping hot pages in RAM makes repeated queries faster.

## Character set (class check)

```
SHOW VARIABLES LIKE 'character_set_server';
-- typically utf8mb4 on modern MySQL
```

`utf8mb4` can use up to **4 bytes per character** (full Unicode, including emoji). That matters when sizing `CHAR` / `VARCHAR` .

## Primary key → automatic index

Creating a PK also creates an index on that column (tree-based internally). Lookups like “is this id present?” become much faster than a full scan. Deeper index theory comes later; for Day 2: **PK ⇒ index ⇒ faster search**.

---

# MySQL Data Types

## Character types: CHAR vs VARCHAR

|                       | CHAR(n)                                 | VARCHAR(n)                                             |
|-----------------------|-----------------------------------------|--------------------------------------------------------|
| Length                | Fixed                                   | Variable up to n                                       |
| Storage               | Always reserves n (pads shorter values) | Stores only actual length (+ length prefix)            |
| Max (practical MySQL) | up to <b>255</b>                        | up to <b>65,535</b> bytes (row limits / charset apply) |
| Use when              | Length always fixed                     | Length varies                                          |

```
-- PIN always 6 digits → CHAR
-- name / email → VARCHAR
name CHAR(20) -- 'Aniket' (6 chars) still occupies 20 (padded)
name VARCHAR(20) -- 'Aniket' occupies ~6 (+ overhead); rest not wasted
```

You still **cannot** store more than the declared `n`. `VARCHAR(20)` rejects a 21-character string.

**Rule of thumb:** `CHAR` for fixed codes (PIN, gender `M / F`); `VARCHAR` for names, emails, cities.

## TEXT family (long character data)

| Type                    | Approx. max size      | Typical use          |
|-------------------------|-----------------------|----------------------|
| <code>TINYTEXT</code>   | 255 bytes             | Very short notes     |
| <code>TEXT</code>       | 65,535 bytes (~64 KB) | Comments, feedback   |
| <code>MEDIUMTEXT</code> | 16 MB                 | Longer articles      |
| <code>LONGTEXT</code>   | 4 GB                  | Very large documents |

No need to declare a length like `VARCHAR(n)` for these.

**Storage note (InnoDB):** small values may sit on the main page with the row. Large TEXT/BLOB payloads use **overflow pages**; the row keeps a **reference/pointer**. Selecting only `id, name` can avoid pulling huge content until you ask for the TEXT column.

```

CREATE TABLE demo_for_text (
 id INT,
 name VARCHAR(20),
 content TEXT
);

SELECT id, name FROM demo_for_text; -- lighter
SELECT id, name, content FROM demo_for_text; -- may fetch overflow pages

```

## BLOB family (binary data)

**BLOB** = Binary Large Object — images, PDFs, audio, video, zip, other binary files.

| Type       | Approx. max size |
|------------|------------------|
| TINYBLOB   | 255 bytes        |
| BLOB       | 65,535 bytes     |
| MEDIUMBLOB | 16 MB            |
| LONGBLOB   | 4 GB             |

Same overflow-page idea as TEXT when the value is large. In many apps, files live on disk/object storage and the table stores only a **path/URL** — but MySQL can store binary in BLOBs when needed.

## ENUM vs SET

Both are a **fixed list of allowed strings**. Difference: how many values per cell.

|                 | ENUM                             | SET                               |
|-----------------|----------------------------------|-----------------------------------|
| Values per cell | <b>Exactly one</b> from the list | <b>Zero or more</b> from the list |
| Example         | Transaction status               | Skills                            |

```

CREATE TABLE enum_demo (
 student_id INT,
 name VARCHAR(20),
 txn_status ENUM('Successful', 'Declined', 'Failed')
);

INSERT INTO enum_demo VALUES (1, 'Aniket', 'Successful');
-- INSERT ... 'Pending' → error / rejection (not in enum list)

CREATE TABLE set_demo (
 student_id INT,
 name VARCHAR(20),
 skills SET('Java', 'MySQL', 'Python', 'CPP')
);

INSERT INTO set_demo VALUES
 (1, 'Aniket', 'Java,MySQL,CPP'); -- multiple skills OK

-- INSERT ... 'MSC' → fails if MSC was never declared in the SET

```

## Integer types (signed / unsigned)

By default integers are **signed** (negative + positive). **UNSIGNED** → only  $\geq 0$  (roughly doubles the positive range).

| Type          | Storage |
|---------------|---------|
| TINYINT       | 1 byte  |
| SMALLINT      | 2 bytes |
| MEDIUMINT     | 3 bytes |
| INT / INTEGER | 4 bytes |
| BIGINT        | 8 bytes |

```
id INT UNSIGNED -- positive ids only
```

## FLOAT vs DOUBLE

|           | FLOAT                 | DOUBLE                      |
|-----------|-----------------------|-----------------------------|
| Size      | 4 bytes               | 8 bytes                     |
| Precision | ~7 decimal digits     | ~15–16 decimal digits       |
| Use       | Approximate values OK | Scientific / high precision |

## BOOLEAN

Stores true/false (internally often `TINYINT(1)` — `1` / `0`).

Examples: marital status, **payment done?**, **seat locked?** (CDAC seat-allocation style flag: click Lock → column flips from false to true).

```
seat_locked BOOLEAN DEFAULT FALSE
```

## Date and time types

| Type      | Format / idea                                                                                      |
|-----------|----------------------------------------------------------------------------------------------------|
| DATE      | YYYY-MM-DD — date difference → days                                                                |
| TIME      | HH:MM:SS (can be wider for durations)                                                              |
| DATETIME  | YYYY-MM-DD HH:MM:SS                                                                                |
| YEAR      | Year value                                                                                         |
| TIMESTAMP | Same wall shape as datetime, but tied to <b>time zone / session</b> (often shown with UTC context) |

**DATETIME vs TIMESTAMP (class takeaway):** both look like date+time, but **TIMESTAMP** carries **session time zone** behaviour; `DATETIME` is a “naive” calendar date-time without that zone conversion story.

```
-- Conceptual examples
-- DATE: '2026-10-09'
-- TIME: '15:30:35'
-- DATETIME: '2026-10-09 15:30:35'
-- TIMESTAMP: '2026-10-09 15:30:35' with time-zone awareness
```

# Day Wrap and Tomorrow

## Covered today — quick checklist

| Topic                      | One-line takeaway                                                       |
|----------------------------|-------------------------------------------------------------------------|
| CMD / Notepad / Workbench  | Exam = CMD; safe-updates can block DELETE in Workbench                  |
| DESCRIBE                   | Null / Key / Default / Extra tell the column story                      |
| INSERT ... SELECT          | Copy projected columns (+ WHERE ) into a working table                  |
| ORDER BY / column list     | Sort and display order are your choice                                  |
| ALTER                      | ADD / MODIFY / CHANGE / DROP COLUMN; RENAME TABLE                       |
| DELETE vs TRUNCATE vs DROP | WHERE? structure? rollback? AI reset?                                   |
| TCL                        | START TRANSACTION / SAVEPOINT / ROLLBACK / COMMIT ; not for DDL         |
| PK + AUTO_INCREMENT        | Unique, not null, auto id; explicit id jumps counter                    |
| DEFAULT / NULL             | Defaults fill gaps; NULL = unknown                                      |
| Temp tables                | Session-scoped sandbox                                                  |
| Operators                  | Comparisons, AND/OR/NOT, IS NULL , IN / NOT IN                          |
| Storage                    | InnoDB pages (16 KB), buffer pool vs disk, utf8mb4                      |
| Data types                 | CHAR/VARCHAR, TEXT, BLOB, ENUM/SET, ints, float/double, bool, date/time |

## Coming next (mentioned, not taught today)

- Foreign keys and table relationships
- Constraints
- Aggregate functions, GROUP BY , HAVING
- LIKE , DISTINCT , more ORDER BY practice
- Joins

Practice today's DDL/DML/TCL on your own schema before tomorrow's keys + joins session.

Coverage: DBT Day 2 meeting (09 Oct 2026) — reconstructed from the class transcript into clean English with runnable MySQL 8 examples and corrected explanations where the ASR was incomplete.